

SECHACK365 RETURNS

WireGuard for Apache NuttX

A WireGuard VPN implementation for Apache NuttX, exposed as a `wg0` network device.

ESP32-S3

TUNNEL UP OVER REAL WI-FI

Real WireGuard peers

LINUX KERNEL, WINDOWS CLIENT

SPRESENSE

WG0 UP — TRAFFIC STILL TO VERIFY

Runtime configuration

SURVIVES A POWER CYCLE

Satoru Akita / Sony Semiconductor Solutions



Apache NuttX



AITRIOS edge AI device (ESP32)



SPRESENSE

SUMMARY

This project implements WireGuard as a NuttX network device.

WHAT IT IS

A modern, lightweight VPN protocol, originally developed for Linux.

WHO MADE IT

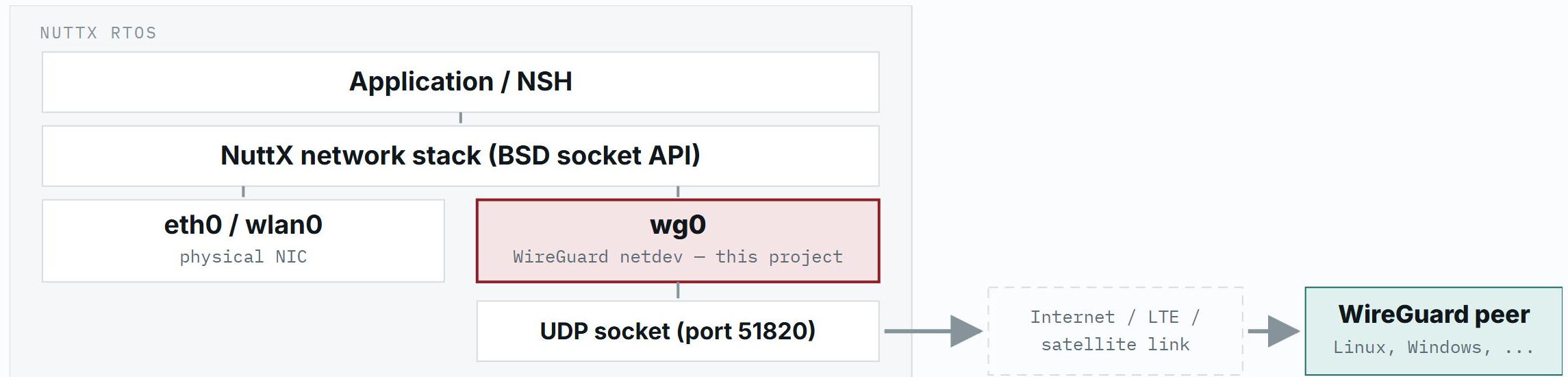
Jason A. Donenfeld, an independent security researcher. First released in **2016**; in the Linux kernel since **5.6 (2020)**.

HOW IT PROTECTS TRAFFIC

Encrypted tunnels over **UDP**: Curve25519, ChaCha20-Poly1305, BLAKE2s.

WHERE YOU HAVE MET IT

Tailscale is built on WireGuard. Roughly **4,000 lines** — small enough for a microcontroller.



THE ASF AND COMMUNITY OVER CODE

Apache NuttX is an ASF project.



"Exists to provide software for the **public good**."

A **501(c)(3) nonprofit organization**, founded in **1999**.

"Run almost exclusively by **volunteers** that provide support for **hundreds of projects**."

Maintains the Apache License 2.0. Its way of working is "The Apache Way" — **community over code**.



"The **official conference** of the Apache Software Foundation (ASF), bringing together the communities behind Apache open source projects."

An annual series for developers, users, contributors, and committers working on ASF projects.

Glasgow: October 11–14, 2026. Sydney: November 18–19.

The NuttX International Workshop is co-located with Glasgow 2026.

That is where this work is presented, and `apache/nuttX-apps` is where the code is to be merged.

WHY THIS TOPIC

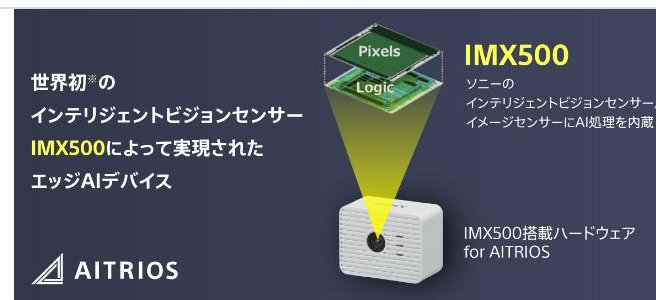
I wanted secure remote access to the NuttX devices I work with.

However, NuttX currently has no native, lightweight VPN capability.

Remote and secure access to NuttX devices is a **real, unsolved problem** across edge AI and industrial IoT, satellite and space hardware, and unmanned infrastructure.

WITHOUT A VPN

Expose a global IP, build a bespoke protocol, or accept a vendor cloud — **each unappealing for its own reasons. So I chose this topic.**



AITRIOS edge AI cameras

ESP32 + NUTTX



SPRESENSE

A BOTTOM-UP SATELLITE PROJECT



The topic itself I found on the Google Summer of Code idea list. **That proposal was not accepted** — the work continued anyway.

THE PLAN

Port an existing implementation: swap its lwIP calls for NuttX ones.

The reference implementation

[smartalock/wireguard-lwip](#) implements WireGuard as an lwIP `netif` — lwIP's abstraction for a **virtual NIC**, treated on equal footing with `eth0` or `wlan0`.

All OS-specific behavior is isolated behind `wireguard-platform.h`: **four functions**. The protocol core and the crypto are **portable C with no OS dependencies**.

What had to be replaced

NuttX does not use lwIP. It has its own TCP/IP stack, and headers such as `lwip/netif.h` are not in the include path — `wireguardif.c` **cannot be compiled on NuttX as-is**.

`struct netif` → `struct net_driver_s`

`pbuf_alloc()` → `iob_alloc()`

`udp_new()` / `bind()` → BSD `socket()` / `bind()`

That was the plan written in the proposal, and that is what the work turned out to be.

The contribution is a **reusable pattern**: how to connect a lwIP-based network component to NuttX's native netdev and socket API.

HOW IT WAS BUILT

Keep the protocol core, replace the network glue.

LAYER	PORTING COST	IN THIS PORT
Protocol and crypto <code>wireguard.c</code> , <code>crypto/</code>	None — portable C, used as-is	3,079 lines byte-identical to upstream
OS hooks <code>wireguard-platform.h</code>	Low — four functions	186 lines written for NuttX
lwIP netif glue <code>wireguardif.c</code>	Total — replaced	2,157 lines <code>nuttX-wireguardif.c</code>

The port moved across four architectures without code changes.

Isolating the OS hooks is why. x86_64 / ARM Cortex-A7 / Xtensa LX7 / ARM Cortex-M4F.

THE DETAILS

`wg0` is a netdev registered with `netdev_register()`, and its wire is a UDP socket.

The data path

TX `devif_poll()` → encrypt → `psock_sendto()`

RX `psock_poll()` → decrypt → `ipv4_input()`

Reception runs in a **background task** that blocks on `psock_poll()`. The device is modelled on `drivers/net/tun.c`.

The four OS hooks

`wireguard_sys_now()` → `clock_gettime(CLOCK_MONOTONIC)`

`wireguard_random_bytes()` → `read("/dev/urandom")`

`wireguard_tai64n_now()` → `clock_gettime(CLOCK_REALTIME)`

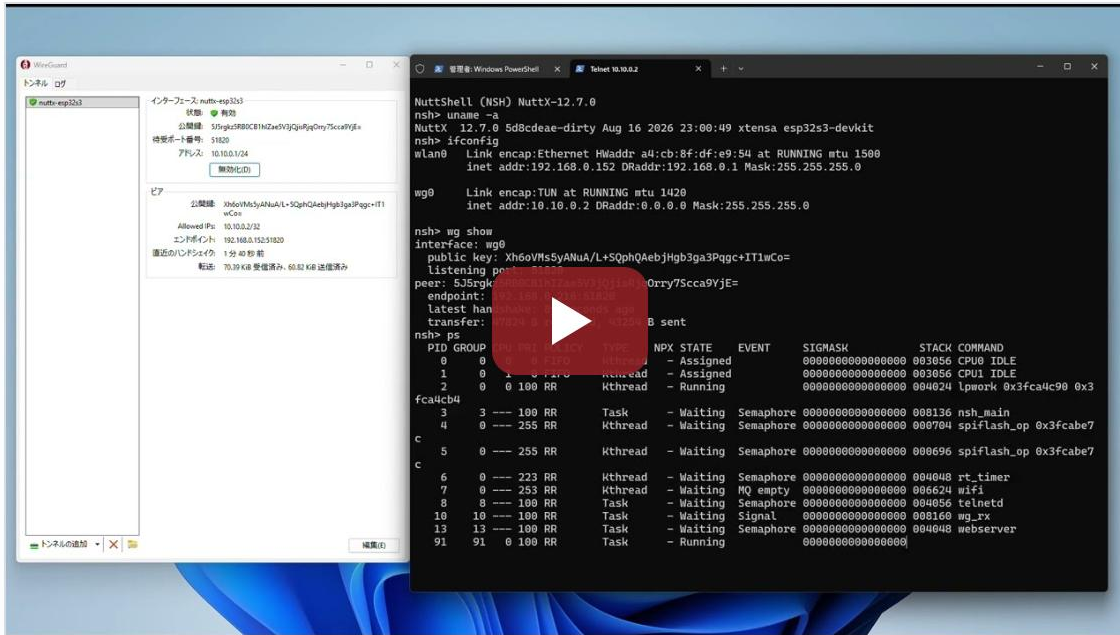
`wireguard_is_under_load()` → `return false`

These four are **all** the reference implementation needs from the platform.

The socket is held as a `struct socket`, not a file descriptor.

One place where the plan did not survive: NuttX scopes descriptors per task group and the transmit path runs on an unrelated worker thread, so `psock_*` replaced the BSD calls. This ties the port to **FLAT builds**.

telnet and a web server, both through the tunnel.



What the demo shows

telnet into the board through the tunnel, and run commands on it.

Start a web server on the board, then open it from a browser.

USB is disconnected. The board runs on a power adapter alone.

The peer is the **official Windows WireGuard client**, over a home Wi-Fi access point.

youtu.be/1kyX2av5WG4

ESP32-S3 · real Wi-Fi · a real WireGuard peer.